**Project Development Phase**
**Model Performance Test**

| | |
|---|---|
| Date | 01.04.2026 |
| Team ID | 69bf8031c1eab70902258949 |
| Project Name | Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets |
| Maximum Marks | |

If you are moving from a ServiceNow environment to Salesforce, the terminology and tools for enforcing field logic and migrating changes are different but follow a similar logic of "Client-side vs. Server-side" and "Staging vs. Production."

In Salesforce, **Validation Rules** and **Dynamic Forms** replace UI Policies, while **Change Sets** or **DevOps Center** serve as the equivalent to Update Sets.

## 1. Enforcing Mandatory Fields (The "UI Policy" Equivalents)

ServiceNow UI Policies are popular because they are conditional and run on the client side. In Salesforce, you have three primary ways to achieve this, depending on how strict you need to be:

| ServiceNow Tool | Salesforce Equivalent | Best Use Case |
|---|---|---|
| **UI Policy (Conditional)** | **Dynamic Forms** | Making a field mandatory **on the UI only** based on another field (e.g., if *Status* is "Closed", then *Close Reason* is mandatory). |
| **UI Policy (Always)** | **Page Layout / Field Level** | When a field should *always* be mandatory for a specific set of users or every time the form is used. |
| **Data Policy / Business Rule** | **Validation Rules** | **The Gold Standard.** Use this to enforce data integrity even if the data comes in via API or Import. It uses a formula like `ISBLANK(Field__c)`. |

**How to set up a Conditional Mandatory Field (Dynamic Forms):**

- Open a record and click the **Gear Icon** > **Edit Page**.
- Click on the **Field** you want to make mandatory.
- In the right-hand sidebar, under **UI Behavior**, select **Required**.
- Under **Set Field Visibility**, add a filter (e.g., `Status = 'Completed'`). This ensures the field only appears and is required when that condition is met.

## 2. Migrating Changes (The "Update Set" Equivalents)

While ServiceNow uses Update Sets to "record" changes, Salesforce uses **Change Sets** (built-in) or **Salesforce DevOps Center** (modern replacement) to "package" them.

**Using Outbound Change Sets:**

- **Step 1: Create:** Go to **Setup** > **Outbound Change Sets** > **New**.
- **Step 2: Add Components:** Unlike Update Sets, Salesforce does not "record" you as you go. You must manually add your components (e.g., the *Custom Field*, the *Validation Rule*, and the *Lightning Record Page*).
- **Step 3: Upload:** Select your target environment (e.g., Production) and click **Upload**.
- **Step 4: Deploy:** Log into the target Org, go to **Inbound Change Sets**, and click **Deploy**.

### Key Difference

In ServiceNow, an Update Set captures everything you do while it is "In Progress." In Salesforce, you build the functionality first, then "gather" the pieces into a Change Set afterward.

### Summary Checklist for Migration

- **Check Dependencies:** If you made a Validation Rule for a *new* field, you must include both the Field and the Rule in your Change Set.
- **Profiles/Permissions:** Unlike ServiceNow, field-level security (FLS) is often handled via Profiles or Permission Sets. Don't forget to include these if you want users to actually see the mandatory fields.
- **Validation Rule Formula:** If you want a field to be mandatory only when a record is saved, use:

  ```
  AND( ISPICKVAL(StageName, "Closed"), ISBLANK(Reason_Field__c) )
  ```

# REFERENCE LINK:

**https://www.youtube.com/watch?t=99&v=hb-4ZV0-xAg**